DIGITAL SYSTEMS LABORATORY

FPGA PONG GAME WITH HDMI OUTPUT

Lab 04 · Assignment 4



OBJECTIVE

Design and implementation of an FPGA-based Pong arcade game with HDMI video output at 640 × 480, 60 Hz, including frame synchronization, sprite rendering, paddle and puck movement, collision handling, and a game restart mechanism.

ANASTASIA EFTHYMIOU – ANDROMACHE SPYROU

03988 – 04059

16 January 2026

Contents

OBJECTIVE 1
SUMMARY & INTRODUCTION 3
Summary 3
Introduction 3
PART A: NEW_FRAME Signal 4
A.1 Implementation 4
A.2 Verification 4
A.3 Experiment and Final Implementation 5
PART B: Block Sprite Rendering 6
B.1 Implementation 6
B.2 Verification 6
B.3 Experiment and Final Implementation 6
PART C: Pong Module and Paddle Movement 7
C.1 Implementation 7
C.2 Verification 7
C.3 Experiment and Final Implementation 8
PART D: Puck Movement and Pong Integration 9
D.1 Implementation 9
D.2 Verification 10
D.3 Experiment and Final Implementation 10
CONCLUSIONS 10
BONUS: Multiplayer Extension 11
Note 11

SUMMARY & INTRODUCTION

Summary

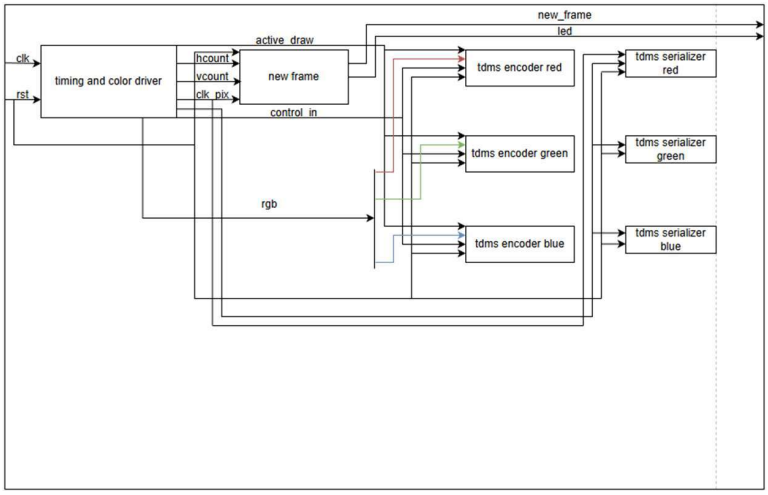
In this project, a Pong arcade game was implemented at a resolution of 640 × 480p and a refresh rate of 60 Hz, using the HDMI controller designed in Assignment 3. The new_frame signal was generated and verified to update the screen after all pixels of the current image had been processed. The paddle and puck were rendered as block sprites by appropriately driving the red, green, and blue color channels, initially at fixed positions. The paddle was then given vertical movement, with the corresponding color outputs updated, while the puck was given diagonal movement. Implementing the latter required handling several collision scenarios involving the walls and the paddle. Finally, the necessary modules were interconnected, the HDMI controller was integrated to drive the display, and the game was completed with appropriate game-over and restart conditions.

Introduction

The objective was to implement a Pong game with a paddle and a puck. The puck moves in different directions, while the paddle moves vertically to return it. Paddle and puck speeds are specified by the paddle_speed and puck_speed signals, respectively, and are controlled from the FPGA board. The HDMI controller provides the vcount and hcount signals for every pixel, allowing the system to determine whether that pixel lies within a sprite (white) or outside the sprites (black). The game ends if the puck reaches the wall on the paddle side. To start another game after game over, the screen must remain static until the new_game signal is asserted. Parameters were introduced for the active-drawing boundaries and the sprite dimensions to make functional verification easier.

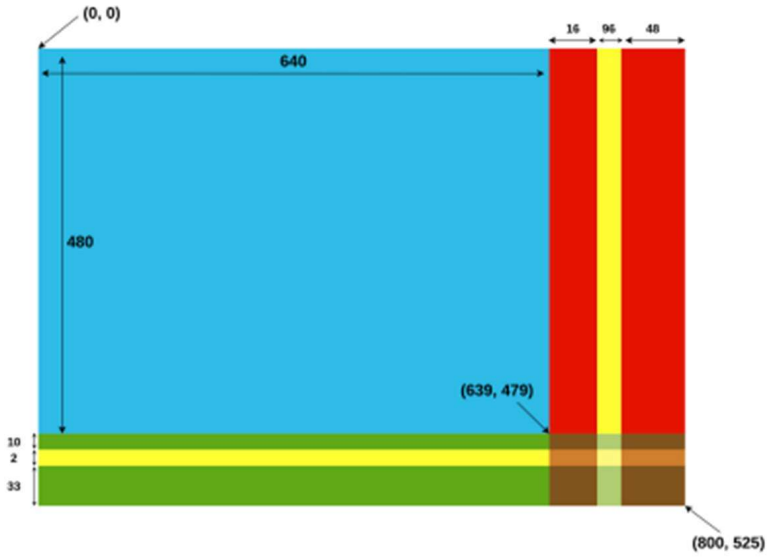
PART A: NEW_FRAME Signal

A.1 Implementation



HDMI architecture with the new-frame generation module.

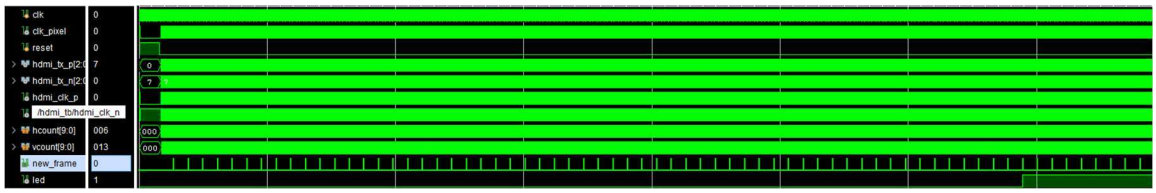
This assignment did not require VRAM to store an image. As a result, the hcount_slow and vcount_slow counters used in the previous FSM for the scaled image were no longer needed. Pixel positions are now defined directly by the 640 × 480p image counters hcount and vcount. To update the displayed frame, the counters are checked so that a new frame is signaled by new_frame upon completion of the active-drawing region. In addition, a count_frame counter was implemented; it increments every time new_frame is asserted, and when it reaches 59, it activates an LED signal intended to blink on the FPGA board.



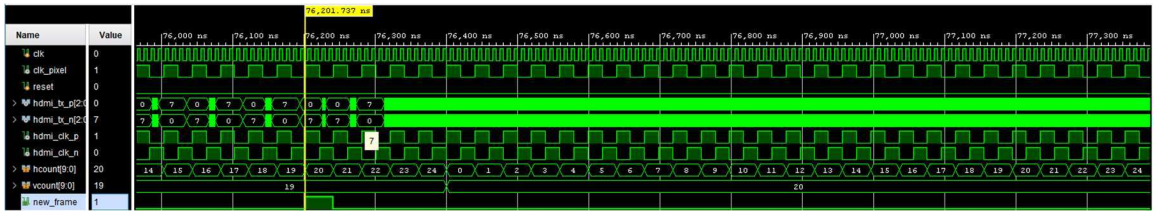
640 × 480 video timing diagram showing the active-drawing area and blanking intervals.

A.2 Verification

A testbench was implemented to drive clk and reset. Inspection of the resulting waveforms showed that new_frame is asserted correctly when hcount and vcount reach 639 and 479, respectively. It was also verified that the LED signal is asserted every 60 new frames.



Simulation of new_frame generation and frame timing.



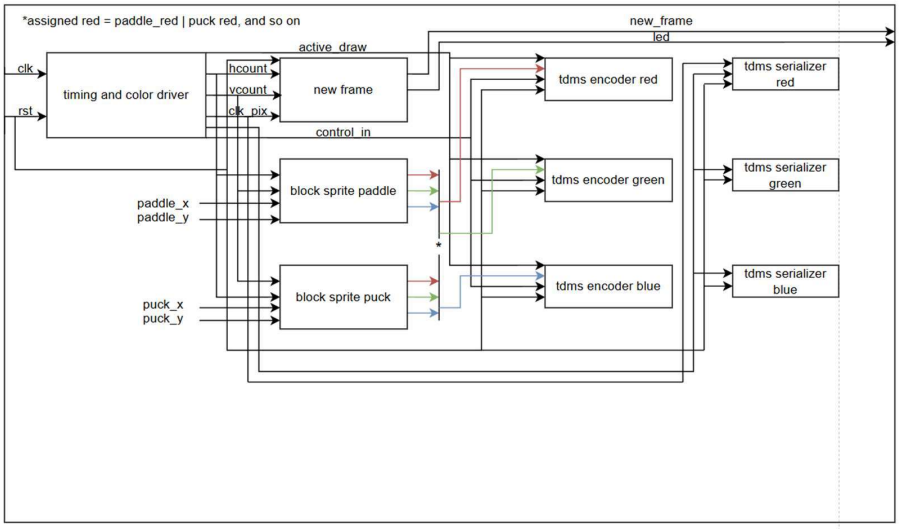
Waveforms confirming LED activation after 60 frames.

A.3 Experiment and Final Implementation

During practical testing of the new_frame module on the FPGA board, the LED was observed to blink once every second, confirming correct operation.

PART B: Block Sprite Rendering

B.1 Implementation



HDMI architecture showing block-sprite modules for the paddle and puck.

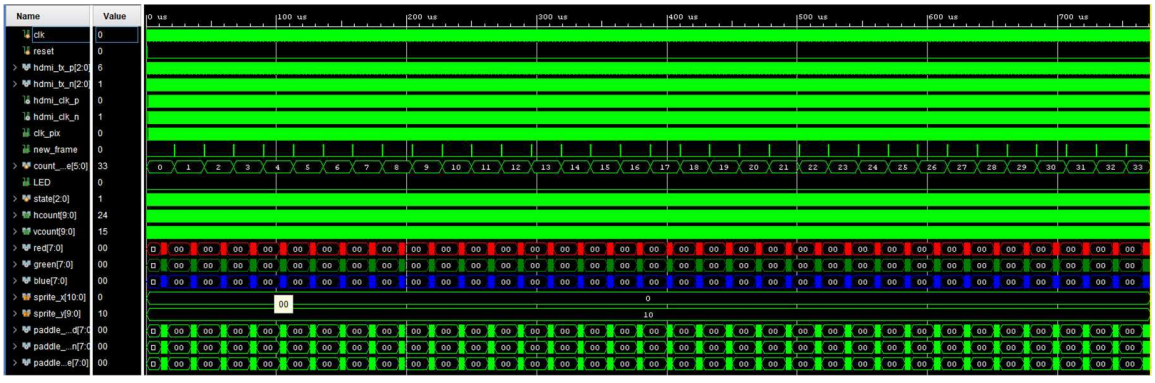
To render the block sprites, a combinational module was developed to determine the color displayed at a given pixel address. When the pixel address lies within the boundaries of a sprite, the module outputs red = 1, green = 1, and blue = 1; otherwise, these outputs are 0. The module is instantiated twice within hdmi.v: once for the paddle and once for the puck.

Parameter	Value
puck_width	32
puck_height	32
paddle_width	10
paddle_height	100

Sprite dimensions in pixels (table recreated in English).

B.2 Verification

A testbench was implemented to simulate the system. The waveforms confirmed that the display color channels were driven correctly.



Testbench waveforms verifying RGB block-sprite rendering.

B.3 Experiment and Final Implementation

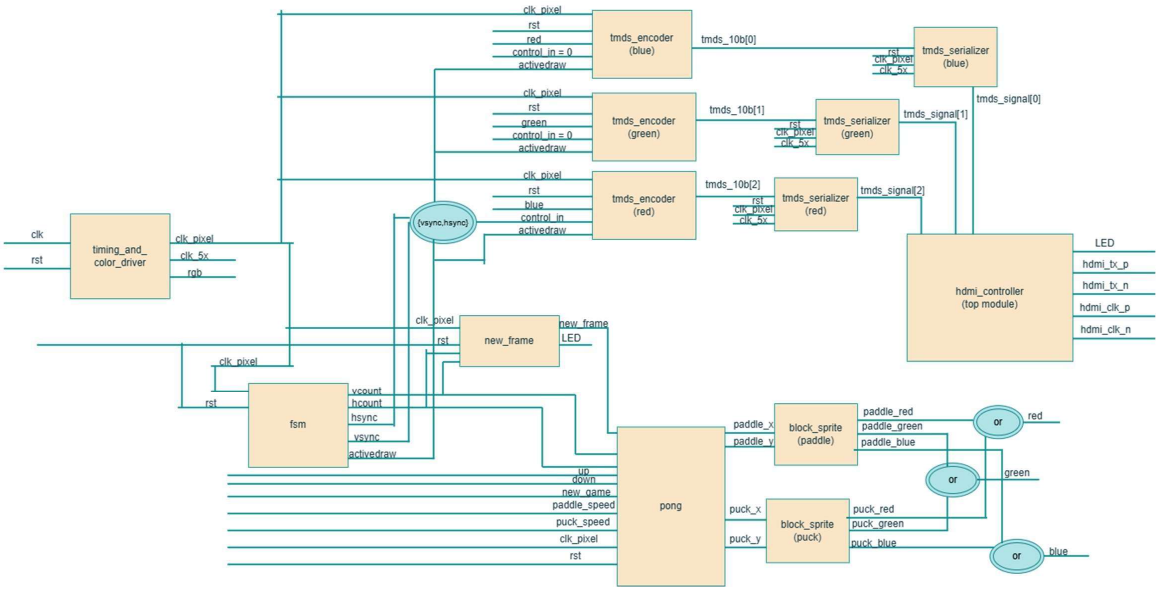
During FPGA testing with the HDMI display, the expected image appeared, with both sprites initialized at the middle of their respective movement ranges.

PART C: Pong Module and Paddle Movement

In this part, vertical movement was implemented for the paddle while the puck remained stationary. The up and down signals, controlled from the FPGA board, determine the paddle direction. Paddle speed is controlled by the board through paddle_speed, measured in pixels per frame.

C.1 Implementation

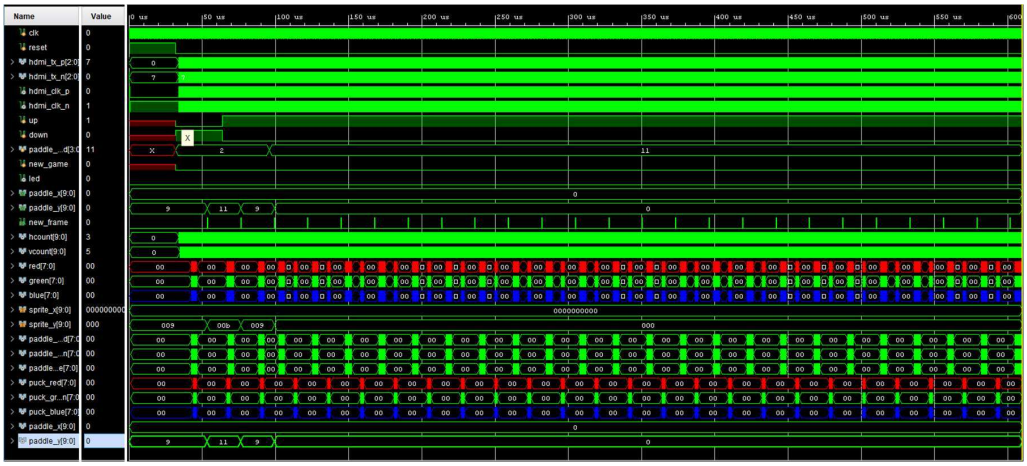
A sequential always block was implemented to check the proposed paddle movement whenever new_frame is asserted. The check uses paddle_speed to determine whether the displacement stays within the frame boundaries. If the requested movement exceeds the upper or lower edge, the paddle moves only as far as that boundary, and the remaining displacement is discarded.



Complete module connections used for paddle movement and HDMI output.

C.2 Verification

Waveforms obtained from the dedicated paddle-movement testbench were examined for different paddle_speed values and up/down inputs. The paddle behaved as expected and never moved outside the defined boundaries.



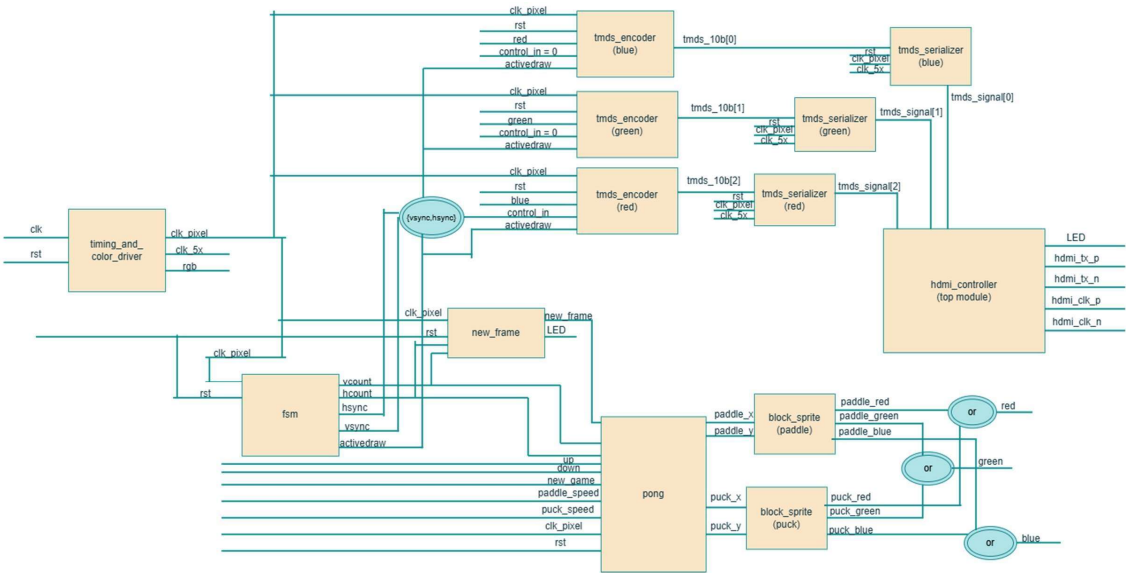
Paddle-movement testbench showing control inputs and position updates.

C.3 Experiment and Final Implementation

Testing the Pong module with paddle movement on the FPGA board confirmed that the paddle always remained within bounds and moved according to the speed and direction selected using the board buttons (up and down). In the event of game over, the image and block sprites remained static, and a new game could be triggered by asserting new_game.

PART D: Puck Movement and Pong Integration

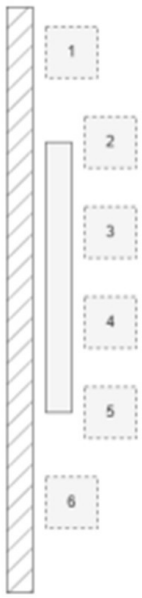
D.1 Implementation



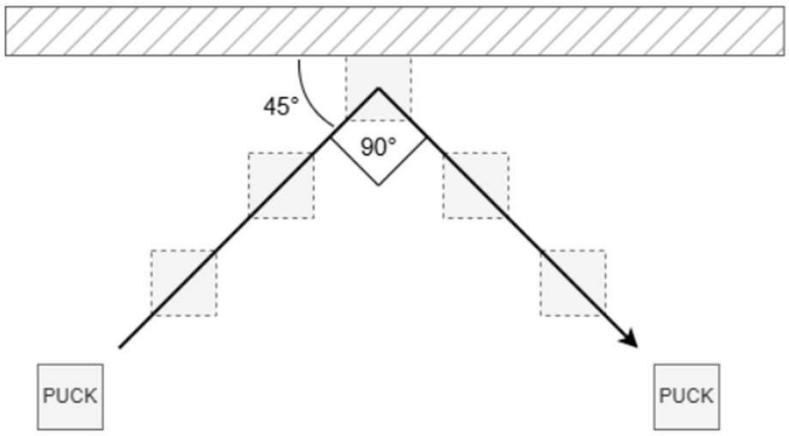
Integrated Pong module with paddle, puck, timing, and HDMI circuitry.

This part completed the Pong module by connecting all previously developed components and the HDMI controller from the preceding assignment. Puck movement limits were defined to prevent the puck from leaving the playfield. If a proposed displacement passes a boundary, the puck is placed at that edge and reflected back into the playfield, except at the left wall, which ends the game.

As shown by the paddle-collision cases, cases 1 and 6 result in game over. In the remaining cases, the puck bounces off the paddle.



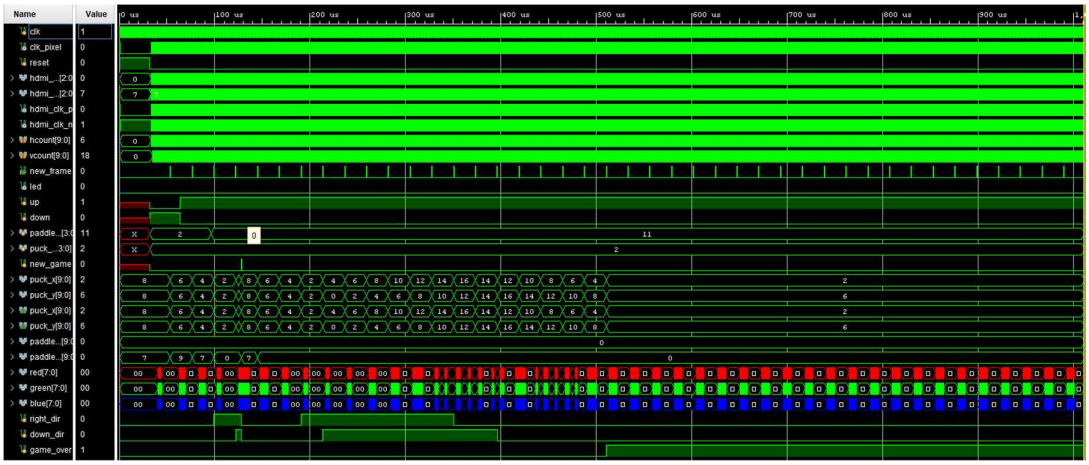
Paddle impact regions and collision cases 1–6.



Puck reflection at the upper wall.

D.2 Verification

To simulate the integrated Pong module, a testbench was developed with suitable puck_speed and paddle_speed values and paddle directions (up and down) at successive time intervals. The waveforms confirmed the operation of the system, the bounded movement of both the paddle and the puck, and the possibility of starting a new game after the previous one ended.



Full-game verification waveform, including paddle and puck movement.

D.3 Experiment and Final Implementation

During practical implementation of Pong with puck movement, the diagonal movement and bounce behavior at the paddle and the walls were confirmed, except at the left wall, where the game ends. An initial issue caused the puck to move beyond the wall boundaries during a bounce. By modifying the collision-handling condition in the code, the puck was kept within the display limits and moved at the speed selected using the board buttons. After game over, the display remained static; pressing the new_game button started another game.

CONCLUSIONS

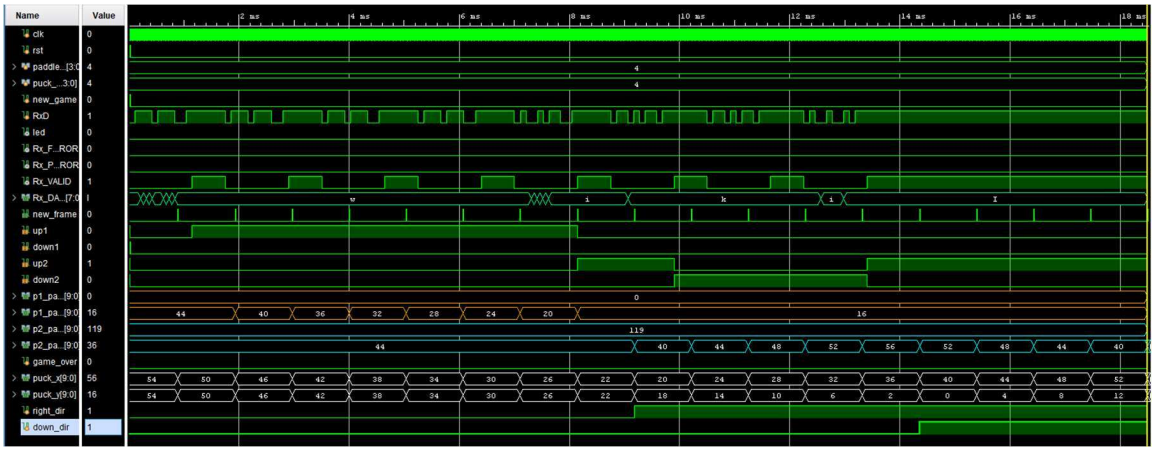
The assignment was completed with the block sprites fully implemented according to the stated specifications and constraints, with particular attention to preventing movement outside the playfield. Challenges arose during development, but they were resolved through detailed testing and a clearer understanding of the system behavior. The main challenge was that a small portion of the puck could incorrectly extend beyond the visible image boundary during reflection because of an error in the collision logic. All difficulties were addressed by

identifying and correcting the errors and thoroughly testing the relevant modules in both simulation and practical FPGA/display operation.

BONUS: Multiplayer Extension

Note

During implementation, an attempt was made to extend Pong to multiplayer operation by incorporating the UART receiver so that commands could be received from an external keyboard or terminal. The approach involved receiving characters and converting them into control signals (up1, down1, up2, down2) to move the paddles and support a second player.



Behavioral simulation of the proposed UART-controlled multiplayer extension.

Although correct behavior was confirmed at the behavioral-simulation level, experimental verification on the FPGA board could not be completed. The evaluation of this extension was therefore limited to simulation results.